

CS105: Python Concurrency, Asyncio & Parallel Programming

Department of Computer Science & Engineering | Academic Handout

Page 1: CPython Internals, the Global Interpreter Lock & Process Models

5.1 CPython Architecture and the Global Interpreter Lock (GIL)

CPython (the reference implementation of Python) employs a Global Interpreter Lock (GIL). The GIL is a mutual-exclusion lock designed to prevent multiple native OS threads from executing Python bytecode simultaneously within a single Python process.

The primary historical justification for the GIL was simplifying memory management: CPython uses reference counting for garbage collection, which is not thread-safe without atomic operations or locking. By locking the interpreter, CPython avoids atomic reference-counting overhead.

Implications of the GIL:

- For CPU-bound tasks (e.g. matrix multiplication, image processing, cryptography), multi-threading in CPython provides NO performance gain and often degrades throughput due to context switching overhead and GIL lock contention.
- For I/O-bound tasks (e.g. network requests, database queries, file read/write), threads release the GIL while waiting for the OS system call to complete, allowing other threads to run concurrently.

5.2 Multithreading vs Multiprocessing in Python

- **threading Module:** Manages native OS threads sharing a single virtual memory space and one CPython interpreter. Lightweight, low memory footprint. Highly effective for concurrent I/O operations.
- **multiprocessing Module:** Spawns independent operating system processes, each running its own CPython interpreter with an isolated memory space and private GIL. Bypasses the GIL entirely, achieving true parallel execution across multi-core CPUs for compute-intensive tasks.

Trade-offs: Processes require Inter-Process Communication (IPC) via Pipes or Queues and necessitate data serialization via pickle, incurring memory and serialization overhead.

Page 2: Asynchronous I/O, Asyncio Architecture & Coroutines

5.3 Asyncio Event Loop Mechanics

Asyncio implements single-threaded cooperative multitasking using an Event Loop.

The Event Loop is an infinite loop that monitors a set of I/O events, timers, and callbacks, multiplexing I/O using native OS primitives (epoll on Linux, kqueue on macOS, I/O Completion Ports [IOCP] on Windows).

- Coroutines: Declared using `async def`. When invoked, they return a coroutine object without executing immediately. Execution starts when awaited or scheduled on the event loop.
- The `await` keyword: Pauses execution of the current coroutine, yielding control back to the event loop so other tasks can execute until the awaited I/O operation finishes.
- `asyncio.create_task(coro)`: Wraps a coroutine into an `asyncio.Task` and registers it immediately on the event loop for concurrent execution.
- `asyncio.gather(*aws)`: Concurrently runs multiple awaitables and gathers their return values in ordered sequence.

5.4 Critical Pitfall: Blocking the Event Loop

Because asyncio operates on a single OS thread, invoking synchronous blocking functions (e.g., `time.sleep()`, `synchronous requests.get()`, or heavy mathematical loops) blocks the entire event loop, preventing all other concurrent coroutines from progressing.

Mitigation Strategies: Use non-blocking async libraries (e.g. `httpx`, `aiofiles`, `asyncpg`), or offload blocking operations to a background thread using `asyncio.to_thread(func, *args)`.

Page 3: Synchronization Primitives, Thread Safety & Executors

5.5 Thread Safety and Race Conditions

Even with the GIL, Python code is NOT automatically thread-safe! The GIL guarantees atomicity only for single C-level bytecode instructions, not composite high-level Python operations.

Example: The statement `counter += 1` translates to three distinct bytecode instructions: `LOAD_GLOBAL`, `INPLACE_ADD`, and `STORE_GLOBAL`. If a thread context switch occurs between `LOAD` and `STORE`, race conditions occur, leading to silent data corruption.

Synchronization Primitives in threading and asyncio:

- Lock: Mutual exclusion lock with `acquire()` and `release()` methods, typically used via context manager `with lock:`.
- RLock (Reentrant Lock): A lock that can be acquired multiple times by the same thread without deadlocking.
- Semaphore: Limits concurrent access to a bounded pool of resources (e.g. limiting simultaneous HTTP client connections to 10).
- Event: Communicates state between threads or tasks; one thread signals `event.set()`, while other threads pause at `event.wait()` until notified.

5.6 High-Level Concurrent Futures: `ThreadPoolExecutor` & `ProcessPoolExecutor`

The `concurrent.futures` module provides a high-level asynchronous execution framework:

- `ThreadPoolExecutor`: Thread pool for concurrent I/O operations, managing worker lifecycle automatically.
- `ProcessPoolExecutor`: Process pool for CPU-bound computations, distributing work across physical CPU cores without manual IPC management.